

React Native: Masters-Level Tutorial

This is for engineers who've shipped production apps and want to understand — and modify — what's happening beneath the JavaScript. We're going into JSI, Fabric, the threading model, Metro internals, and the tooling used to run React Native at scale.

1. The Threading Model, Precisely

React Native actually runs on several threads, and knowing which does what explains most "why is this janky" questions:

- **JS Thread** — runs your JavaScript, including render logic and event handlers. Only one at a time; if it's busy, touch events and animations driven from JS stall.
- **UI Thread (Main Thread)** — the native thread that actually draws views (`UIView` on iOS, `View` on Android). This is what must stay at 60/120fps.
- **Shadow Thread** — runs the Yoga layout engine, calculating flexbox layout off the UI thread so layout math doesn't block drawing.
- **Native Modules Thread** — where TurboModule/legacy native module calls execute by default, so slow native calls (disk I/O, crypto) don't block JS.

Before JSI, all cross-thread communication was **serialized to JSON and batched over an async bridge** — even calling a native function meant marshaling arguments to a string, queuing a message, and waiting for a batched response. This is why old-architecture apps had visible lag on rapid interactions like scroll-linked animations.

JSI changes this: JS objects can hold direct references (`HostObject`s) to C++ objects, and calls can be **synchronous** when needed, with no serialization. Reanimated's worklets exploit this directly — a "worklet" is a JS function whose bytecode is copied and executed on the UI thread via JSI, so gesture-driven animations never touch the JS thread at all.

2. Writing a TurboModule in C++/JSI

Beyond Expo Modules (which wrap this for you), you can write a JSI host object directly for maximum control — useful for performance-critical native code shared across platforms.

```
// MyMathModule.h
#include <jsi/jsi.h>
using namespace facebook::jsi;

class MyMathModule : public HostObject {
public:
    Value get(Runtime& runtime, const PropNameID&
name) override {
        auto propName = name.utf8(runtime);

        if (propName == "fibonacci") {
            return Function::createFromHostFunction(
```

```

        runtime,
        name,
        1, // arg count
        [](Runtime& rt, const Value&, const
Value* args, size_t count) -> Value {
            int n = (int)args[0].getNumber();
            long a = 0, b = 1;
            for (int i = 0; i < n; i++) { long t =
a + b; a = b; b = t; }
            return Value((double)a);
        }
    );
}
return Value::undefined();
}
};

```

Installed into the runtime at startup:

```

runtime.global().setProperty(
    runtime,
    "MyMath",
    Object::createFromHostObject(runtime,
std::make_shared<MyMathModule>())
);

```

From JS, this is now just `global.MyMath.fibonacci(30)` — a direct, synchronous C++ call, no bridge, no serialization, no thread hop for the call itself. This is the mechanism most performance-critical libraries (Reanimated, MMKV, Skia) actually use under the hood.

3. Fabric: Building a Custom Native UI Component

Fabric replaces the old view manager system with a C++ shadow tree shared across platforms. Writing a Fabric component means defining it once and implementing rendering natively per platform.

Step 1 — Codegen spec (TypeScript describing the component's props):

```
// specs/MyGaugeNativeComponent.ts
import type { ViewProps } from 'react-native';
import type { Int32 } from 'react-native/Libraries/Types/CodegenTypes';
import codegenNativeComponent from 'react-native/Libraries/Utilities/codegenNativeComponent';

export interface NativeProps extends ViewProps {
  value: Int32;
  maxValue: Int32;
}

export default
  codegenNativeComponent<NativeProps>('MyGauge');
```

Codegen reads this at build time and generates the C++ props structs, Java/ObjC interfaces, and shadow node classes — you never hand-write the bridging boilerplate that older View Managers required.

Step 2 — Native implementation (iOS, Swift/ObjC++ side conceptually):

```
@implementation MyGaugeView

- (void)updateProps:(Props::Shared const &)props
```

```

        oldProps:(Props::Shared const
&)oldProps
{
    const auto &newProps =
*std::static_pointer_cast<const MyGaugeProps>
(props);
    self.progressLayer.value = newProps.value;
    self.progressLayer.maxValue =
newProps.maxValue;
    [super updateProps:props oldProps:oldProps];
}

@end

```

The key architectural shift: **props update through an immutable C++ struct diffed on the shadow thread**, and only the delta is applied natively — this is what enables Fabric's "single commit" rendering versus the old architecture's multiple bridge round-trips per update.

4. Yoga: The Layout Engine

Every `flex`, `padding`, and `justifyContent` you write compiles down to a call into **Yoga**, Meta's cross-platform C++ implementation of Flexbox. Understanding it helps you debug layout issues that don't match web intuition:

- Yoga defaults `flexDirection` to `column` (web defaults to `row`) — this is the single most common web-to-RN layout surprise.
- `flex: 1` in RN maps to `flexGrow: 1`, `flexShrink: 1`, `flexBasis: 0%` — not just `flex-grow` like on web.
- Percentage-based dimensions require the parent to have a resolved size; an unbounded `ScrollView`

content container with `height: '100%'` children is a classic bug because Yoga can't resolve it.

You can measure and influence layout imperatively when needed:

```
const ref = useRef(null);

const measure = () => {
  ref.current.measure((x, y, width, height,
    pageX, pageY) => {
    console.log({ width, height, pageX, pageY
  });
});
};

<View ref={ref} onLayout={(e) =>
  console.log(e.nativeEvent.layout)} />
```

`onLayout` fires after Yoga resolves a node's layout on the shadow thread and commits it — useful for measuring dynamic content before animating it.

5. Metro Internals

Metro is RN's bundler — conceptually similar to webpack but built for RN's constraints (fast rebuilds via caching, platform-specific resolution).

Custom resolvers

```
// metro.config.js
const { getDefaultConfig } =
  require('expo/metro-config');
```

```
const config = getDefaultConfig(__dirname);

config.resolver.resolveRequest = (context,
moduleName, platform) => {
  // Redirect all imports of 'lodash' to
  'lodash-es' for tree-shaking
  if (moduleName === 'lodash') {
    return context.resolveRequest(context,
    'lodash-es', platform);
  }
  return context.resolveRequest(context,
  moduleName, platform);
};
```

Custom transformers

Transformers run on every file Metro processes (e.g., stripping flow types, applying Babel). A custom transformer can, for example, inline SVGs as components:

```
config.transformer.babelTransformerPath =
require.resolve('react-native-svg-transformer');
config.resolver.assetExts =
config.resolver.assetExts.filter((ext) => ext
!== 'svg');
config.resolver.sourceExts =
[...config.resolver.sourceExts, 'svg'];
```

Bundle splitting for large apps

Metro supports **RAM bundles** and, more relevantly today, **Hermes bytecode precompilation** — Hermes compiles JS to bytecode at build time rather than parsing JS on-device at startup, which is the single biggest lever for cold-start time

on large apps. This is on by default with Hermes; verify it's active via `android/app/build.gradle`'s `hermesEnabled` flag or the Expo `jsEngine` config.

6. Monorepos and Module Federation

At organizational scale, teams often split RN apps into independently deployable JS bundles.

Monorepo structure (Nx or Turborepo)

```
apps/  
  mobile-app/      # the RN app shell  
packages/  
  ui-components/   # shared design system  
  api-client/      # shared data layer  
  feature-checkout/ # feature module
```

Turborepo config for caching builds across packages:

```
{  
  "pipeline": {  
    "build": { "dependsOn": ["^build"],  
    "outputs": ["dist/**"] },  
    "test": { "dependsOn": ["build"] }  
  }  
}
```

Module Federation with Re.Pack

Re.Pack swaps Metro for a webpack/Rspack-based toolchain, enabling true **Module Federation** — separate

teams ship independently versioned bundles that a host app loads at runtime:

```
// rspack.config.mjs (host app)
import { ModuleFederationPlugin } from
 '@callstack/repack/webpack';

export default {
  plugins: [
    new ModuleFederationPlugin({
      name: 'host',
      remotes: {
        checkout:
'checkout@https://cdn.example.com/checkout/mobile.bundle',
      },
    }),
  ],
};
```

```
// Host app dynamically loading the remote
const CheckoutScreen = React.lazy(() =>
import('checkout/CheckoutScreen'));
```

This is genuinely advanced territory — reach for it only when you have multiple teams needing independent release cycles for a single app; the operational complexity (versioning, shared dependency alignment, remote bundle caching) is substantial.

7. Advanced Native Debugging & Profiling

iOS: Instruments

- **Time Profiler** — sample stacks across all threads to find hot native/JS-bridge paths.
- **Allocations** — track retain cycles; RN apps commonly leak via closures capturing `this` in event listeners that are never removed.
- **Core Animation** — visualize offscreen rendering and view flattening issues (common cause: `shouldRasterize` misuse or excessive shadow/blur views).

Android: Android Studio Profiler + Perfetto

- **CPU Profiler** — trace method calls across the JS, UI, and native modules threads simultaneously; look specifically for long JS-thread frames correlating with dropped UI frames.
- **Memory Profiler** — watch for native heap growth from Image caches or un-released native views (common with maps/video components).
- **Perfetto traces** — Hermes can emit trace events (`SysTrace`) that show up alongside native traces, letting you correlate a JS function call with the exact native frame it caused.

Hermes-specific tooling

```
# Capture a sampling profile directly from
Hermes
npx react-native profile-hermes ./profiles
# Convert to Chrome DevTools format for
flamegraph viewing
```

```
npx hermes-profile-transformer
./profiles/<file>.cpuprofile
```

8. App Size and Startup Optimization

- **Hermes bytecode** is smaller and faster to parse than JS source — confirm it's enabled (default in modern RN/Expo).
- **Enable Proguard/R8** (Android) and **bitcode stripping/App Thinning** (iOS) to cut binary size for unused code paths.
- **Split by ABI** on Android (`arm64-v8a` , `armeabi-v7a`) so users don't download architectures they don't need — Google Play App Bundles handle this automatically.
- **Image assets**: ship `@2x` / `@3x` variants appropriately and prefer WebP over PNG where transparency isn't essential — this is often the largest binary size contributor.
- **Startup**: measure Time-To-Interactive with tools like `react-native-startup-time` ; the largest wins are usually deferring non-critical `useEffect` work (analytics init, non-visible data prefetch) until after first paint.

9. Bridgeless Mode

The New Architecture's endpoint is **bridgeless mode**: no async bridge exists at all — JSI is the only communication path, even for legacy interop. This matters because:

- Any native module NOT yet migrated to TurboModules needs an interop layer, and older libraries can break in subtle ways (synchronous assumptions, missing lifecycle hooks) until they're updated.
- Startup is measurably faster since there's no bridge initialization step.
- Debugging tools built around bridge message inspection (some older Flipper plugins) stop working; Hermes' native debugger protocol (CDP) is the forward-compatible path.

When auditing a large app for New Architecture migration, the actual work is almost always in third-party library compatibility, not your own code — check `directory.reactnative.dev` for each dependency's New Architecture support status before starting.

Where This Leads

At this level, the highest-leverage next steps are usually organizational rather than purely technical:

- Contribute to or read the source of libraries you depend on (Reanimated, MMKV, FlashList) — their C++/JSI implementations are the best real-world reference for these patterns.
- Set up automated New Architecture compatibility checks in CI as your dependency graph grows.
- If you're building a platform (not just an app), study how Expo's own EAS/Metro tooling is structured — it's one of

the most mature examples of RN tooling at scale in the open.